



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: objective metric that scores how successfully an agent's actions modify the environment state
E – Environment: the external world state, contexts, and systems the agent inhabits
A – Actuators: physical/software mechanisms the agent triggers to execute actions

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.agent_pos, self.walls, self.food_positions, self.opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

`self.opponents` other rational entities trying to maximize their own performance scores, not just static elements. collides with the main agent, the score is reduced. Since there are multiple agents acting in the environment and their actions affect each other, it is a multi-agent environment.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

Percept Sequence: the absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment this is central to the Russell & Norvig definition of AI itself

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` represents the Sensors (S) component of the PEAS framework because it provides information from the environment to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

sensory limits hide parts of the state. so the agent's sensors do not detect all relevant environment states at every decision instance. rational agent is never omniscient — it can only maximize expected utility based on the clues actually present in its percept sequence, not the full unseen truth.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The performance measure is updated because the agent's score changes after hitting a wall.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

performance measures must evaluate changes in the external state of the environment, NOT the internal processing state of the agent. This connects directly not by internal thought processes, since "we can measure if a car crashes, but we cannot measure if it felt fear." Measuring internal effort instead of external outcome

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Stays Partially Observable — sensory limits/occlusions hide parts of the state; the traps exist in the environment but are hidden from the agent's sensors, so it cannot detect all relevant states. this doesn't make the agent "irrational" for stepping on a hidden trap — rationality is bounded by what is perceivable, not by omniscient

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

maximizing the mathematical expected utility based strictly on the clues present in the perceived history up to that point. Lecture 01 defines a rational agent as one that "acts to achieve the best possible outcome, or under uncertainty, the best expected outcome based on its current knowledge baseline." Adding `'smells_toxin'` expands

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit: an agent given +1 point per unit of dirt collected will "clean a patch of dirt, purge its container back onto the floor, and clean the exact same spot repeatedly forever to harvest infinite score points" — a rational

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING